

LABORATORY

Project No. 1

Financial Fraud Detection

Classification of Highly Imbalanced Data

Report No. 2

Project Implementation and Evaluation Metrics

Submitted by:

Mykola Tatarenkov (53631)

Siarhei Kryshchanovich (57763)

1. Introduction

This report presents the implementation of a fraud-detection workflow for highly imbalanced binary classification problems. In such tasks, fraudulent transactions constitute only a very small fraction of all observations, yet the operational consequences of missing them are severe. For this reason, the project was designed not as a simple model-training script, but as a structured experimental pipeline that integrates data loading, preprocessing, imbalance handling, model comparison, threshold tuning, cross-validation, and cost-sensitive evaluation.

The repository is configured for two datasets with different characteristics: the anonymized Credit Card Fraud Detection dataset and the more business-oriented PaySim transaction dataset. This dual-dataset design improves the study because it tests the methodology on two different feature structures: one dominated by transformed numerical variables and one containing interpretable transactional attributes. As a result, the report can evaluate not only model performance, but also the robustness of the implementation across different fraud-detection settings.

A central idea of the project is that default machine-learning practice is not sufficient for fraud detection under extreme class imbalance. Accuracy alone is misleading, the default probability threshold of 0.5 is rarely optimal, and false negatives are substantially more costly than false positives. Therefore, the implemented workflow emphasizes PR-AUC, recall, expected cost, validation-based threshold tuning, and experiment automation.

2. Project Implementation

2.1 Datasets and targets

The experiment pipeline supports two datasets defined in the configuration module. For the **creditcard** dataset, the target column is Class. For the **paysim** dataset, the target column is isFraud. This dataset-specific target handling is centralized in the code so that the rest of the workflow remains dataset-agnostic.

The **PaySim** dataset also requires additional methodological precautions. The columns nameOrig and nameDest behave like identifiers rather than meaningful predictive features, so they are excluded from the feature matrix. In addition, isFlaggedFraud is excluded by default and can be enabled only through a command-line flag. This is a deliberate safeguard because the variable may encode post-decision information and could artificially inflate performance if used without caution.

Dataset	Target	Special handling	Reason
creditcard	Class	No identifier-like exclusions required	Features are already anonymized numerical variables
paysim	isFraud	Drop nameOrig, nameDest; exclude isFlaggedFraud by default	Reduce leakage risk and avoid non-informative identifiers

2.2 Data loading and feature selection

Dataset loading is implemented in a dedicated module, which resolves file paths, loads CSV files, extracts the configured target, converts labels to integer form, and constructs the feature matrix after dropping the target and any dataset-specific excluded columns. This design is important for reproducibility because all dataset-specific decisions are concentrated in one place rather than scattered across the project.

From an experimental perspective, this separation is valuable because it reduces the risk of accidental inconsistencies between runs. Every model receives data that was prepared according to the same rules, which makes later comparisons between models and imbalance strategies more trustworthy.

2.3 Preprocessing strategy

Preprocessing is implemented with a reusable ColumnTransformer. Numeric features are processed through median imputation followed by standard scaling, while categorical features are processed through most-frequent imputation and one-hot encoding. In the PaySim dataset, the transaction type field is explicitly treated as categorical, and any other categorical columns are handled through the same generic pipeline.

This preprocessing design is methodologically strong for two reasons. First, it standardizes data preparation across all models, which supports fair comparisons.

Second, preprocessing is embedded inside the model pipeline, so fitting occurs only on the training data. This helps prevent data leakage, since scaling and encoding are not learned from validation or test subsets.

2.4 Train/validation/test split

The experiment logic uses a two-stage stratified split. First, the full dataset is divided into `train_temp` and test subsets with a 80/20 split. Next, `train_temp` is split again into train and validation subsets, where validation receives 25% of `train_temp`. The effective proportions are therefore approximately 60% train, 20% validation, and 20% test.

Stratification is essential here because both datasets are highly imbalanced. Without stratification, the minority class could be distributed unevenly across subsets, which would make performance estimates unstable and threshold tuning unreliable. The dedicated validation set also plays an important methodological role: it is used to select the best threshold without contaminating the final test evaluation.

2.5 Imbalance handling strategies

The implementation evaluates six strategies for handling class imbalance: baseline, `class_weight`, random undersampling, random oversampling, SMOTE, and ADASYN. Sampling-based approaches are inserted into imbalanced-learn pipelines, while the `class_weight` strategy adjusts the model itself rather than changing sample counts. In the case of XGBoost, the class-weight idea is implemented through `scale_pos_weight` estimated from the observed training class ratio.

The key methodological point is that balancing occurs only during training. Validation and test sets remain untouched so that the reported metrics reflect realistic class proportions. This is one of the most important safeguards in the project, because applying resampling before the split or outside the training fold would create leakage and overstate model performance.

2.6 Implemented models

The project includes five model families: Logistic Regression, Decision Tree, Random Forest, Extra Trees, and XGBoost. This set was selected to provide a meaningful comparison across different modeling assumptions. Logistic Regression serves as a linear baseline; Decision Tree acts as an interpretable tree-based baseline; and Random Forest, Extra Trees, and XGBoost provide stronger ensemble alternatives capable of modeling more complex decision boundaries.

This model selection contributes to the experimental rigor of the study. The report is not based on a single algorithm, but on a structured comparison across linear, tree-based, and ensemble approaches. That is particularly important in fraud detection, where non-linearity, rare-event structure, and cost asymmetry often interact in ways that cannot be captured by a single baseline model.

2.7 Threshold tuning and expected cost

A particularly important part of the implementation is threshold tuning on validation data. Instead of assuming that 0.5 is an acceptable operating point, the project evaluates thresholds from 0.01 to 0.99 and selects the one that minimizes expected cost. The threshold sweep for each experiment is exported to a dedicated CSV file, making the selection process transparent and reproducible.

Expected cost is computed from the confusion matrix using asymmetric penalties: $COST_{FP} = 10$ and $COST_{FN} = 500$. This means that a false negative is treated as fifty times more expensive than a false positive. Such a formulation is consistent with the practical logic of fraud detection, where missing a fraud case usually causes more damage than temporarily flagging a legitimate transaction.

2.8 Output generation and experiment artifacts

The project generates multiple types of outputs. For each experiment, threshold-sweep CSV files are saved, together with ROC curves, precision-recall curves, and tuned-threshold confusion matrices. When the pipeline completes, consolidated comparison and hyperparameter-analysis tables are also saved.

Beyond static files, the repository also includes an **HTML dashboard** (**`fraud_detection_analysis_dashboard.html`**) that reads directly from the outputs folder. The dashboard can scan the generated CSV files dynamically and visualize current experiment results without hardcoding the datasets, models, or strategies. This strengthens the practical usability of the project and supports reproducibility because results can be explored interactively after the experiments finish.

3. Most Important Implemented Functions

3.1 main.py

The most central function is *run_all_experiments(...)*. It orchestrates the full experimental workflow by iterating over datasets, models, and balancing strategies; performing stratified splitting; building preprocessing and training pipelines; fitting models; tuning thresholds on validation data; evaluating the final operating point on test data; running cross-validation; and collecting the final results rows.

Methodologically, this is the backbone of the project because it guarantees that all experiments follow the same logic.

The helper *_run_hyperparameter_analysis(...)* performs compact randomized hyperparameter search with cross-validation. Its role is not to exhaustively optimize every model, but to provide model-selection evidence through a controlled search guided primarily by PR-AUC. This supports the claim that the project includes not only fixed-parameter comparisons, but also a more advanced tuning stage.

The function *run_hyperparameters_only(...)* adds practical flexibility. It allows the tuning stage to be executed independently of the full experiment pipeline, which is especially useful when the main plots and comparison outputs have already been generated. This also improves reproducibility because one expensive stage can be rerun without repeating all earlier steps.

Finally, *_build_cv_scores()* centralizes the scoring dictionary for cross-validation, while *_parse_args()* defines command-line controls for dataset selection, optional inclusion of *isFlaggedFraud*, and hyperparameter-only mode. These functions are small, but they matter because they keep the workflow consistent and transparent.

3.2 config.py

The configuration file defines the global experimental scope: split sizes, cost parameters, dataset metadata, model lists, imbalance strategies, and hyperparameter-analysis settings. It is particularly important because it makes the project reproducible at the configuration level. Changing the experiment space does not require rewriting the main workflow; instead, the user modifies centralized settings and reruns the pipeline.

A strong feature of the configuration design is the presence of dataset-specific constraints for hyperparameter analysis. In particular, the PaySim dataset uses reduced folds, restricted strategies, subsampling, and lower parallelism during tuning. This is a methodologically defensible compromise: it preserves structured tuning logic while keeping the computation feasible on local hardware.

3.3 data_loader.py and preprocessing.py

The function *load_dataset(...)* is crucial because it determines exactly what information enters the model. It resolves paths, loads the data, extracts the target,

and removes inappropriate columns. In a fraud-detection project, this is more than a technical step: it directly affects leakage control and study validity.

The function *build_preprocessor(...)* creates a dataset-aware ColumnTransformer for numeric and categorical features. Its importance lies in standardization: every model is trained on features prepared under the same transformation logic, which supports fair comparison across algorithms and strategies.

3.4 models.py

The function *get_base_models()* defines the set of estimators used in the experiments, while *get_hyperparameter_grid(model_name)* defines compact search spaces for tuning. The function *get_sampler(strategy)* maps abstract strategy names to actual imbalanced-learn resamplers, and *_apply_class_weight(...)* handles model-specific weighting logic, including *scale_pos_weight* for XGBoost.

The most important function in this module is *build_pipeline(...)*, which assembles preprocessing, optional sampling, and the classifier into a single imbalanced-learn pipeline. This function is methodologically essential because it ensures that preprocessing and resampling are fitted only on the training data within each experiment or cross-validation fold.

3.5 evaluation.py and utils.py

The evaluation module contains *expected_cost_from_cm(...)*, *compute_metrics(...)*, *tune_threshold(...)*, *save_curves(...)*, and *save_confusion_matrix(...)*. Together, these functions transform raw probabilities into fraud-detection diagnostics that are meaningful from both statistical and operational perspectives.

The utility functions *ensure_dir(path)* and *experiment_name(dataset_name, model_name, strategy)* are simple but important for reproducibility. They provide deterministic naming and reliable creation of artifact directories, which helps keep outputs traceable and organized across large experiment runs.

4. Program Execution Flow

The execution flow can be described as a controlled experimental pipeline rather than a simple sequential script. First, the program parses command-line arguments and determines which datasets should be processed. Then the selected dataset is loaded, the target is extracted, and any restricted columns are excluded.

After that, the dataset is split into train, validation, and test subsets with stratification preserved throughout. Next, the code enters nested loops over the configured models and imbalance strategies. For each combination, it builds the appropriate preprocessor, constructs the pipeline, trains the training subset only, and obtains validation probabilities.

The validation predictions are then passed to the threshold-tuning routine, which scans candidate thresholds and selects the one minimizing expected cost. Only after the threshold has been chosen on validation data does the workflow evaluate the model on the untouched test subset. This sequence is crucial: it separates model-selection logic from final assessment and prevents optimistic bias.

At the same time, the workflow saves threshold tables, ROC curves, precision-recall curves, and confusion matrices. In parallel, a cross-validation pipeline is evaluated on `train_temp` to obtain mean and standard deviation estimates for the main metrics. When enabled, the project also runs a separate hyperparameter-analysis stage that produces candidate-level tuning outputs.

This execution logic demonstrates experimental rigor because it formalizes the full process: controlled data preparation, train-only balancing, validation-based threshold optimization, untouched test evaluation, and saved artifacts for later inspection.

5. Evaluation Metrics in Fraud-Detection Context

5.1 Accuracy, Precision, and Recall

Accuracy is reported in the project but not treated as the main decision metric. In heavily imbalanced fraud data, a classifier can achieve deceptively high accuracy by labeling almost everything as legitimate. Such a result would be operationally useless because it would fail to capture the minority class that matters most.

Precision measures how many predicted fraud cases are truly fraudulent. From an operational perspective, this reflects alert quality: low precision means many false alarms and greater manual review burden. Recall, by contrast, measures how many actual fraud cases are detected. In fraud detection, recall is often more critical because false negatives correspond to missed fraud and direct loss. The tension between these two metrics is one of the main trade-offs in the project.

5.2 F1-score, ROC-AUC, and PR-AUC

F1-score summarizes the balance between precision and recall at a fixed threshold. It is useful as a compact threshold-dependent indicator, but it does not replace deeper analysis of the precision-recall trade-off.

ROC-AUC measures ranking performance across thresholds in terms of true-positive and false-positive rates. It is useful as a global discrimination metric, but it can appear overly optimistic in highly imbalanced settings because even a low false-positive rate may still translate into many false alarms in absolute terms.

PR-AUC is therefore more informative in this project. It evaluates the precision-recall trade-off across thresholds and focuses more directly on minority-class detection quality. For this reason, PR-AUC is a primary model-ranking metric in the hyperparameter search and a key interpretive metric in the results section.

5.3 Confusion Matrix and Expected Cost

The confusion matrix provides the counts of true negatives, false positives, false negatives, and true positives at a selected threshold. These counts are directly interpretable in operational terms and form the basis for cost analysis.

Expected cost combines the confusion matrix with asymmetric penalties. In this project, the cost formula reflects the assumption that false negatives are much more expensive than false positives. This is why threshold tuning is based on expected cost rather than a generic metric like accuracy. In a real fraud-detection setting, the best operating point is not the one that maximizes raw correctness, but the one that minimizes the expected business damage.

6. Summary of Current Experimental Results

For the creditcard dataset, the strongest cost-sensitive result was obtained by Random Forest with the class_weight strategy. The tuned threshold was 0.17, with ROC-AUC equal to 0.978639, PR-AUC equal to 0.846396, recall equal to 0.887755, precision equal to 0.587838, and expected cost reduced from 11150 at threshold 0.5 to 6110 at the tuned threshold. The corresponding cross-validation PR-AUC mean and standard deviation were 0.828605 and 0.007042, which indicates a strong and relatively stable configuration.

For the PaySim dataset, the strongest cost-sensitive result was obtained by XGBoost with SMOTE. The tuned threshold was 0.87, with ROC-AUC equal to 0.999830, PR-AUC equal to 0.961160, recall equal to 0.986602, precision equal to 0.493601, and expected cost reduced from 49740 at threshold 0.5 to 27620 at the tuned threshold. The cross-validation PR-AUC mean and standard deviation were 0.922526 and 0.003036, which suggests high ranking quality together with stable validation performance.

Dataset	Best model	Strategy	Threshold	PR-AUC	Recall	Precision	Expected cost
creditcard	Random Forest	class_weight	0.17	0.846396	0.887755	0.587838	6110
paysim	XGBoost	smote	0.87	0.961160	0.986602	0.493601	27620

Several general conclusions emerge from these results. First, ensemble models dominate the top rows, which suggests that the fraud patterns in both datasets benefit from model families capable of capturing richer non-linear structure than simple baselines. Second, PR-AUC is more informative than accuracy for ranking useful fraud-detection systems, because it reflects how well the model identifies rare fraud cases without hiding behind majority-class dominance.

Third, threshold tuning changes the practical operating point in a substantial way. For both datasets, the tuned threshold lowers expected cost compared with the default threshold of 0.5. This shows that model performance in fraud detection cannot be interpreted solely from raw probabilities or default settings; operational objectives must be imposed explicitly.

Finally, low cross-validation standard deviations support the reliability of the best-performing configurations. Stability does not guarantee perfect generalization, but it strengthens confidence that the strong results are not driven only by one favorable split.

7. Methodological Correctness

Methodological correctness is one of the strongest aspects of the project. The implementation avoids the most common sources of optimistic bias by maintaining a strict separation between training, validation, and test data. The validation set is used for threshold selection, while the test set is reserved for final evaluation. This means that the reported test performance is not directly optimized during the threshold-tuning process.

The project also addresses data leakage in a disciplined way. Preprocessing and balancing are implemented inside reusable pipelines, so transformations are learned only on the relevant training data. Sampling methods are never applied globally before the split, and validation and test targets remain untouched. These decisions are critical in imbalanced learning, where even small leakage can produce misleadingly strong results.

PaySim requires additional methodological caution. Identifier-like columns are removed, and `isFlaggedFraud` is excluded by default because of its potential to encode information too close to the operational decision itself. This demonstrates awareness that fraud-detection datasets often contain features that look convenient but may not be suitable for honest predictive modeling.

The hyperparameter-analysis design for PaySim is also methodologically defensible despite being lighter than the creditcard tuning stage. Because PaySim is computationally heavier, the project uses reduced folds, subsampling, limited strategy sets, and restricted parallelism for tuning. This does not invalidate the analysis; rather, it reflects a realistic compromise between methodological structure and available computational resources.

8. Reproducibility and Experimental Rigor

The project provides a strong level of reproducibility. The codebase is modular, with separate files for configuration, data loading, preprocessing, models, evaluation, and utility functions. This structure makes the workflow easier to inspect, modify, and rerun. In addition, the configuration file centralizes the experiment space, so models, strategies, costs, and hyperparameter settings can be reviewed and changed in a transparent way.

Deterministic output naming further supports reproducibility. Threshold tables, plots, confusion matrices, comparison files, and hyperparameter-analysis outputs are generated according to stable naming conventions derived from dataset, model, and strategy names. This allows users to trace each artifact back to the corresponding experiment.

The project also demonstrates experimental rigor through automation. Instead of manually running selected models, the workflow systematically iterates over all configured datasets, models, and balancing strategies. Cross-validation, threshold tuning, cost evaluation, and artifact generation are integrated into the same pipeline, which reduces ad hoc decisions and improves comparability between experiments.

The HTML dashboard strengthens the practical side of reproducibility. Because it reads directly from the outputs folder and discovers available experiment files dynamically, it makes the generated artifacts easier to inspect after the run. This does not replace the main analysis, but it improves traceability and interpretability of results.

9. Critical Analysis and Study Limitations

A major strength of the project is that it does not equate strong fraud detection with high accuracy. Under severe class imbalance, accuracy can be achieved too easily by favoring the majority class. In contrast, this project treats model evaluation as an objective-sensitive problem: if the objective is minority-class ranking, PR-AUC becomes crucial; if the objective is operational risk, expected cost becomes central. As a result, model ranking may change depending on the criterion being optimized.

This leads directly to the precision-recall trade-off. A model with very high recall may identify more fraud, but it may also generate more false positives. Conversely, a model with higher precision may miss too many fraud cases. Threshold tuning is therefore not a cosmetic step; it is the mechanism that translates probability outputs into a practical operating point consistent with the cost structure.

The best threshold is also not fixed at 0.5. In both datasets, the tuned threshold differs substantially from the default value, and the expected-cost reduction is large. This shows that the threshold itself is part of the model-selection problem in fraud detection. Reporting only the classifier and its default threshold would miss an important part of the practical solution.

There are, however, clear study limitations. The PaySim hyperparameter analysis is computationally expensive, which required a lighter tuning configuration with reduced folds, subsampling, and restricted parallelism. While this is justified for feasibility, it means that the tuning results for PaySim are less exhaustive than they could be under unlimited resources. In addition, the minority class is rare in both datasets, which always creates uncertainty and makes metrics more sensitive to thresholding and split variation.

Another limitation is that synthetic oversampling methods such as SMOTE and ADASYN may not behave equally well across all feature structures. They can improve minority representation, but they also introduce synthetic points that may not perfectly reflect the true data-generating process. This does not invalidate their use; it simply means that their results should be interpreted critically rather than accepted automatically.

Future work could strengthen the project further through broader hyperparameter searches on more powerful hardware, probability calibration, richer cost scenarios, additional ensemble methods, or evaluation closer to deployment conditions. Nevertheless, the current implementation already demonstrates a mature applied pipeline with clear methodological safeguards and meaningful operational analysis.

10. Conclusion

This report has shown that the project implements a complete and methodologically structured fraud-detection workflow for highly imbalanced data. The repository supports two datasets, five model families, multiple imbalance-handling strategies, validation-based threshold tuning, cross-validation, cost-sensitive evaluation, hyperparameter analysis, and systematic output generation.

More importantly, the project shows a substantive experimental lesson: in fraud detection, the model alone is not enough. The final system quality depends on the interaction between preprocessing, imbalance strategy, model family, threshold choice, and the objective used to judge success. PR-AUC, recall, and expected cost provide a much more realistic basis for comparison than accuracy alone.

The strongest observed results support the value of ensemble methods, especially when combined with explicit cost-sensitive threshold selection. At the same time, the report shows awareness of methodological risks such as leakage, over-optimistic evaluation, and computational limitations. For these reasons, the implemented workflow should be considered not only a working classification project, but a reproducible and critically analyzed fraud-detection experiment pipeline suitable for a high-level university submission.